

GUÍA DE LABORATORIO: OPTIMIZACIÓN EN LA JERARQUÍA DE MEMORIA

Análisis Empírico de Líneas de Caché (64B), Registros de CPU y Paralelismo ILP en Linux

Asignatura: Arquitectura de Computadoras (SIS-131)

Entorno: Ubuntu 24.04 LTS (x86_64) | Intel(R) Core(TM) i7-13620H

1. Comandos de Terminal Ejecutados y Salidas Verificadas

1.1 Verificación Previa de Topología de Memoria y Línea de Caché (Paso 2)

```
javier@localhost:~$ lscpu | grep -E "L1|L2|L3|Model name"
```

Model name:	13th Gen Intel(R) Core(TM) i7-13620H
L1d cache:	192 KiB (6 instances)
L1i cache:	384 KiB (6 instances)
L2 cache:	12 MiB (6 instances)
L3 cache:	144 MiB (6 instances)
Vulnerability L1tf:	Not affected

```
javier@localhost:~$ getconf LEVEL1_DCACHE_LINESIZE  
64
```

```
javier@localhost:~$ cat /sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size  
64
```

1.2 Compilación Determinística Oficial con -O1

```
javier@localhost:~/Documents$ gcc -Wall -Wextra -O1 benchmark_arquitectura.c -o  
benchmark_arquitectura -lm
```

1.3 Ejecución del Benchmark y Medición de Rendimiento

```
javier@localhost:~/Documents$ ./benchmark_arquitectura
```

=== RESULTADOS DETERMINISTICOS (N=1024) ===

```
1. Naive (i-j-k)      : 3.9156s | 0.55 GFLOPS | Speedup: 1.00x
2. Localidad Espacial (Cache) : 0.5805s | 3.70 GFLOPS | Speedup: 6.75x
3. Registros de CPU    : 0.6062s | 3.54 GFLOPS | Speedup: 6.46x
4. Loop Unrolling 4x (ILP)  : 0.4303s | 4.99 GFLOPS | Speedup: 9.10x
[OK] Validacion de Checksum: 1.5839e+09 (Error = 0.0000e+00)
```

1.4 Perfilado de Eventos del Sistema con perf

```
javier@localhost:~/Documents$ sudo sysctl kernel.perf_event_paranoid=-1
kernel.perf_event_paranoid = -1
```

```
javier@localhost:~/Documents$ perf stat -e
task-clock,context-switches,cpu-migrations,page-faults,cycles,instructions
./benchmark_arquitectura
```

=== RESULTADOS DETERMINISTICOS (N=1024) ===

```
1. Naive (i-j-k)      : 3.8947s | 0.55 GFLOPS | Speedup: 1.00x
2. Localidad Espacial (Cache) : 0.6299s | 3.41 GFLOPS | Speedup: 6.18x
3. Registros de CPU    : 0.5796s | 3.71 GFLOPS | Speedup: 6.72x
4. Loop Unrolling 4x (ILP)  : 0.4593s | 4.68 GFLOPS | Speedup: 8.48x
[OK] Validacion de Checksum: 1.5839e+09 (Error = 0.0000e+00)
```

Performance counter stats for './benchmark_arquitectura':

```
5564.32 msec task-clock
109 context-switches
0 cpu-migrations
6212 page-faults
<not supported> cycles
<not supported> instructions
```

```
5.576260350 seconds time elapsed
5.535682000 seconds user
0.029478000 seconds sys
```

2. Plantilla de Registro de Métricas (Punto 6)

Fase / Configuración	Tiempo Medido (s)	Rendimiento (GFLOPS)	Aceleración (Speedup)	Tasa Acierto Caché
----------------------	-------------------	----------------------	-----------------------	--------------------

1. Naive (i-j-k)	3.8947 s	0.55 GFLOPS	1.00x (Base)	Baja (< 15%)
2. Localidad Espacial (i-k-j)	0.6299 s	3.41 GFLOPS	6.18x	Alta (> 90%)
3. Uso de Registros CPU	0.5796 s	3.71 GFLOPS	6.72x	Óptima
4. Loop Unrolling 4x (ILP)	0.4593 s	4.68 GFLOPS	8.48x	Máxima

3. Cuestionario de Análisis Crítico (Punto 7)

Pregunta 1 (Líneas de Caché)

Enunciado: Si una línea de caché mide 64 bytes y cada float ocupa 4 bytes, ¿cuántos accesos a memoria consecutivos aprovechan una sola carga a la caché L1 en la Fase 2? Demuestre la fórmula teórica del Hit Rate.

Respuesta y Demostración:

1. Capacidad de la Línea de Caché:

Dado que una línea física de la memoria caché L1d mide 64 Bytes (LEVEL1_DCACHE_LINESIZE = 64) y cada dato de tipo float ocupa 4 Bytes (sizeof(float) = 4), la cantidad de elementos almacenados por línea de transferencia es:

$$\text{Elementos por línea} = \frac{64 \text{ Bytes}}{4 \text{ Bytes}} = 16 \text{ floats}$$

2. Aprovechamiento Consecutivo:

En la Fase 2 (i-k-j), los elementos de la matriz $B[k][j]$ y de $C[i][j]$ se acceden contiguamente incrementando el índice j .

- En el primer acceso ($j = 0$), se produce un fallo forzoso de caché (**Cache Miss**), transfiriendo el bloque completo de 64 Bytes (16 floats) desde los niveles inferiores de memoria hacia L1d.
- Los siguientes **15 accesos consecutivos** ($j = 1, 2, \dots, 15$) ya residen en la línea cargada, resolviéndose como aciertos directos (**Cache Hits**).
- Por tanto, **16 accesos consecutivos** aprovechan una sola carga a la caché (1 Miss

inicial seguido de 15 Hits).

3. Demostración de la Fórmula Teórica del Hit Rate:

$$\text{Hit Rate} = \left(\frac{\text{Accesos con Acierto (Hits)}}{\text{Total de Accesos a la Línea}} \right) \times 100$$

$$\text{Hit Rate} = \left(\frac{16 - 1}{16} \right) \times 100 = \frac{15}{16} \times 100 = 93.75\%$$

En contraste, en la Fase 1 (i-j-k), cada iteración de k salta una fila completa de B ($1024 \times 4 \text{ Bytes} = 4096 \text{ Bytes}$), excediendo con creces los 64 Bytes de la línea y desaprovechando el 93.75% de los datos transferidos.

Pregunta 2 (Localidad Temporal y Registros)

Enunciado: En la Fase 3, ¿qué ventaja física representa almacenar `reg_a` en un registro de la FPU/ALU en lugar de releerlo desde la memoria en cada paso del bucle interior?

Respuesta:

1. Latencia Física de Acceso:

Un registro del procesador forma parte integral de la ruta de datos (*datapath*) de la CPU. Su tiempo de acceso es de **1 ciclo de reloj (latencia prácticamente despreciable, < 1 ns)**. La memoria caché L1d, aunque extremadamente veloz, introduce una latencia típica de **4 a 5 ciclos de reloj**, mientras que la memoria RAM principal requiere entre **150 y 200 ciclos**.

2. Descongestión del Bus y de los Puertos de Carga (Load Ports):

En la Fase 2, la instrucción interna requería emitir dos operaciones de carga (*load*) en cada iteración del bucle j : una para leer $A[i][k]$ y otra para $B[k][j]$. Al almacenar el valor invariante en un registro mediante `register const float reg_a = A[i*N + k]`, se elimina por completo la lectura repetida de A durante las 1024 iteraciones del bucle j .

3. Optimización de Recursos:

Se liberan las unidades de generación de direcciones (AGU) y los puertos de lectura de la caché L1, permitiendo que el subsistema de memoria atienda exclusivamente la transferencia secuencial del vector $B[k][j]$ y la acumulación sobre $C[i][j]$.

Pregunta 3 (Loop Unrolling e ILP)

Enunciado: ¿Por qué el desenrollado de bucles en la Fase 4 reduce el tiempo de ejecución incluso cuando el número total de sumas y multiplicaciones matemáticas es exactamente el mismo?

Respuesta:

El volumen total de operaciones de punto flotante se mantiene invariante (

$2 \times 1024^3 \approx 2.147 \times 10^9$ operaciones FLOP). La aceleración (alcanzando **4.68**

GFLOPS y un Speedup de **8.48x**) se debe a factores microarquitectónicos:

1. Reducción del Overhead de Control del Bucle:

Al avanzar de cuatro en cuatro ($j += 4$), las instrucciones de control asociadas al bucle (incremento del contador `add`, comparación de límite `cmp` y salto condicional `jl`) se reducen en un **75%**. Menos ciclos de reloj se desperdician en la administración del flujo de control.

2. Explotación del Paralelismo a Nivel de Instrucción (ILP):

Las cuatro instrucciones generadas en el cuerpo desenrollado:

```
ptr_c[j] += reg_a * ptr_b[j];  
ptr_c[j + 1] += reg_a * ptr_b[j + 1];  
ptr_c[j + 2] += reg_a * ptr_b[j + 2];  
ptr_c[j + 3] += reg_a * ptr_b[j + 3];
```

operan sobre posiciones contiguas pero independientes en memoria y registros. Los procesadores modernos superescalares y fuera de orden (*Out-of-Order Execution*), como el Intel Core i7-13620H, disponen de múltiples puertos de ejecución y unidades FMA (*Fused Multiply-Add*). Al no existir dependencias de datos cruzadas entre estas cuatro operaciones, el procesador despacha y ejecuta múltiples instrucciones en paralelo en una misma ventana de ciclos.

3. Mitigación de Paradas en el Pipeline (*Pipeline Stalls*):

Al espaciar las operaciones, se oculta la latencia de las operaciones de lectura y multiplicación de la FPU, saturando de forma continua las tuberías de segmentación sin bloquear los registros temporales.

Pregunta 4 (Validación de Determinismo)

Enunciado: ¿Por qué el valor del Checksum matemático debe ser idéntico en las cuatro fases? ¿Qué indicaría si el Checksum de la Fase 4 difiere del de la Fase 1?

Respuesta:

1. Identidad Matemática y Consistencia Algorítmica:

La multiplicación matricial $C = A \times B$ es una operación determinística que satisface las propiedades distributiva y asociativa. Las cuatro variantes computacionales

implementadas corresponden al mismo modelo matemático:

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} \cdot B_{k,j}$$

Dado que las cuatro funciones procesan los mismos arreglos de entrada A y B bajo la misma precisión (IEEE 754 de 32 bits), la suma total de todos los elementos calculados ($\sum C_{i,j}$) debe arrojar exactamente el mismo resultado (Checksum: 1.5839e+09).

2. **Determinismo Numérico en las Mediciones:**

La ejecución confirmó:

$$\text{Error} = |chk_1 - chk_4| = 0.0000e + 00$$

Esto certifica que no hubo alteración de precisión por redondeo acumulativo ni fallos de acceso a datos.

3. **Significado de una Divergencia en el Checksum de la Fase 4:**

Si el checksum de la Fase 4 difiriese del de la Fase 1, constituiría una evidencia crítica de:

- **Error de desbordamiento de punteros o índices (*Out-of-bounds Access*):** Al iterar con paso $+4$, un cálculo incorrecto de los límites (N) provocaría escritura o lectura en memoria no reservada.
- **Falta de inicialización o condiciones de carrera:** Si la matriz destino C no se limpiara a cero ($idx < N*N$; $C[idx] = 0.0f$) antes del cómputo, se acumularían valores residuales de memoria ("basura").
- **Pérdida de asociatividad en punto flotante:** Modificaciones no controladas en el orden de reducción asociativa de números con magnitudes dispares que generen cancelación catastrófica.